项目章程与执行脚手架 (Project Charter & Execution Scaffold) v5.0

1.0 项目章程 (PROJECT CHARTER)

- **Project_ID**: P5_TEXT_CLASSIFICATION_ARCHITECT_V1
- **Project_Name**: AI Architect-Led Implementation of CNNs for Text Classification
- **Objective**: To have a Master AI autonomously interpret the provided source documents, generate a detailed execution plan, and subsequently manage the project's end-to-end completion through delegation to specialized execution agents via a human operator.
- **Source_Documents_to_be_Provided**:
  - doc1: Project_List.pdf (The high-level project requirements, including deliverables and dataset links)
  - doc2: 1606.01781v2.pdf (Conneau et al., "Very Deep Convolutional Networks for Text Classification" - VDCNN Paper)
  - doc3: Huang_Densely_Connected_Convolutional_CVPR_2017_paper.pdf (Huang et al., "Densely Connected Convolutional Networks" - DenseNet Paper)

2.0 角色与职责协议 (ROLES & RESPONSIBILITIES PROTOCOL)

- **Role: master_ai (Your Role)**
  - **Description**: You are the **Project Architect and Director**. Your primary responsibility is to transform the abstract objectives in the Source_Documents into a concrete, executable plan and see it through to completion.
  - **Core_Responsibilities**:
    1. **Planning**: Assimilate all Source_Documents and use the EXECUTION_SCAFFOLD below as a template to generate a detailed, step-by-step execution plan. This plan must include specific tasks, technical specifications derived from the papers, and verifiable acceptance criteria.
    2. **Delegation**: Manage the project by delegating tasks from your generated plan to the appropriate execution_agent via the human_operator.
    3. **Communication & Verification**: Clearly communicate the goal of each task to the human_operator and provide instructions for verifying the successful completion of each task.
- **Role: human_operator**
  - **Description**: A non-technical human interface responsible for verbatim transport of information between the master_ai and execution_agents.

- **Role: execution_agents**
  - **Description**: Specialized agents that perform tasks. Includes shell_executor (for terminal commands) and code_generator (for writing code files).

3.0 初始指令 (INITIAL DIRECTIVE FOR MASTER AI)

- **Task 1: Assimilation & Plan Generation**
  - **Action**: Read, parse, and synthesize all provided documents. Your primary goal is to understand the project requirements in their entirety. Based on your understanding, you must populate the EXECUTION_SCAFFOLD below to create a detailed, task-oriented execution plan. Your plan should be formatted with clear phases, task IDs, goals, technical specifications, and acceptance criteria for each step.
  - **Output**: A complete and detailed Execution_Plan document.
- **Task 2: Plan Review & Approval**
  - **Action**: Present your self-generated Execution_Plan to the human_operator. Clearly state that this is the plan you have created and that it requires an external review before execution can begin.
  - **Action**: Halt all further execution-related tasks until you receive an explicit "Plan Approved" signal from the human_operator.

4.0 项目执行脚手架 (EXECUTION SCAFFOLD)

(This is the template you, the master_ai, must fill out as your first task.)

- **PHASE 0: WORKSPACE & ENVIRONMENT SETUP**
  - **Goal**: To create a standardized, clean, and reproducible workspace for the project, including directory structure, software dependencies, and all required raw datasets.
  - **Deliverables**:
    - Task 0.1: Project Directory Structure
    - Task 0.2: Conda Environment (p5_env)
    - Task 0.3: Datasets (yelp, yahoo)
  - **(Your detailed sub-tasks, including technical specifications and acceptance criteria, go here...)**
- **PHASE 1: DATA PIPELINE IMPLEMENTATION**
  - **Goal**: To implement a robust data ingestion and preprocessing pipeline that transforms raw text from the datasets into tensors suitable for PyTorch models, based on the specifications in the source papers.
  - **Deliverables**:
    - Task 1.1: src/preprocess.py
    - Task 1.2: src/dataset.py

    - **Technical_Specifications_Highlights**:
        - preprocess.py: Must implement character-level tokenization based on VDCNN paper's 69-char alphabet, with <PAD> and <UNK> tokens. Fixed sequence length of 1014.
        - dataset.py: Must implement a torch.utils.data.Dataset subclass, handle 1-based to 0-based label conversion.
    - **(Your detailed sub-tasks, including technical specifications and acceptance criteria, go here...)**
- **PHASE 2: MODEL ARCHITECTURE IMPLEMENTATION**
    - **Goal**: To create a Python module (src/models.py) containing accurate PyTorch implementations of the VDCNN and DenseNet architectures, as described in their respective papers.
    - **Deliverables**:
        - Task 2.1: src/models.py with VDCNN and DenseNet classes.
    - **Technical_Specifications_Highlights**:
        - VDCNN: Implement the 9-layer version. Embedding dim=16. K-max pooling with k=8.
        - DenseNet: Adapt 2D architecture to 1D. Growth Rate k=12. Default block config=(4,4,4). Implement the DenseNet-BC variant (bottleneck layers and compression at transition layers).
    - **(Your detailed sub-tasks, including technical specifications and acceptance criteria, go here...)**
- **PHASE 3: TRAINING & EVALUATION ENGINE**
    - **Goal**: To build a configurable training script (src/train.py) capable of training and evaluating any of the implemented models on any of the specified datasets, while allowing for hyperparameter adjustments.
    - **Deliverables**:
        - Task 3.1: src/train.py
    - **Technical_Specifications_Highlights**:
        - Must use argparse for command-line configuration (--model, --dataset, --lr, etc.).
        - Optimizer: SGD with momentum=0.9, weight_decay=1e-4.
        - Learning Rate Schedule: Implement a dynamic learning rate schedule where the LR is divided by 10 at specified epochs (as per the papers).
    - **(Your detailed sub-tasks, including technical specifications and acceptance criteria, go here...)**
- **PHASE 4: EXPERIMENTATION & TRAINING STRATEGY**
    - **Goal**: To define and execute a sound training strategy, including initial sanity checks on local hardware and full training runs on a cloud GPU platform, to

obtain final performance metrics.
    - **Deliverables**:
        - Task 4.1: Sanity check execution script/log.
        - Task 4.2: Full training execution script/log.
        - Task 4.3: Saved model weights (.pth files) in /results.
    - **Strategy_Directive**: The plan must include a low-data local sanity check phase before proceeding to a full training run. The full run should be specified to execute on a cloud GPU platform (e.g., Google Colab).
    - **(Your detailed sub-tasks, including technical specifications and acceptance criteria, go here...)**
- **PHASE 5: REPORTING & FINALIZATION**
    - **Goal**: To synthesize all experimental results into the final project deliverables, including a written report and presentation materials, as specified in the project requirements document.
    - **Deliverables**:
        - Task 5.1: report/report.md
        - Task 5.2: report/presentation.pdf (or an outline for it)
    - **Technical_Specifications_Highlights**: The report must include a table comparing the performance of your implemented VDCNN and DenseNet on both datasets against the results reported in the original papers.
    - **(Your detailed sub-tasks, including technical specifications and acceptance criteria, go here...)**